

Application Web de Gestion des notes personnelles

ENSEIGNANT : HAMDI NASREDINE

REALISE PAR : YASSAMINE FENNY

1. Introduction

Ce projet consiste à développer une application web complète de gestion de notes personnelles, réalisé dans le cadre du module de Programmation Web. L'objectif est de mettre en pratique l'intégration d'un backend API REST (Laravel) avec un frontend dynamique (React), en exploitant une authentification sécurisée par token.

L'application permet à chaque utilisateur authentifié de créer, consulter, modifier et supprimer ses propres notes, chacune associée à un niveau de priorité (Basse, Moyenne, Haute). Toutes les interactions se font via des requêtes AJAX, sans rechargement de page.

Fonctionnalité	Description
Inscription / Connexion	Création de compte et authentification sécurisée via Sanctum
Gestion des notes (CRUD)	Ajout, modification, suppression et affichage des notes
Priorités colorées	Badge vert (Basse), orange (Moyenne), rouge (Haute)
Filtre par priorité	Filtrage instantané sans rechargement de page
Retours visuels	Notifications toast après chaque action CRUD

2. Architecture Technique

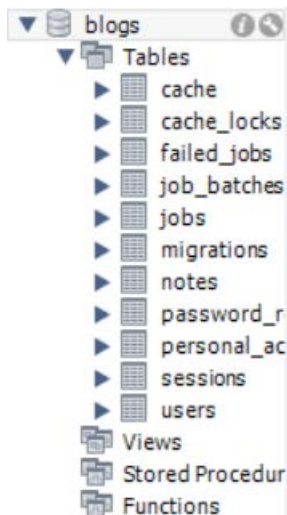
Couche	Technologie	Rôle
Backend	Laravel 10/11	API REST, logique métier, validation
Authentification	Laravel Sanctum	Génération et révocation de tokens
Base de données	MySQL / SQLite	Persistence des utilisateurs et notes
Frontend	React 18 (Vite)	Interface dynamique, composants fonctionnels
Communication	Axios	Requêtes HTTP AJAX entre React et Laravel
Style	Bootstrap / Tailwind CSS	Interface responsive et moderne

L'architecture suit un modèle client-serveur découplé en trois couches :

React (Frontend) → Requête HTTP/AJAX (Axios) → API Laravel (Backend) → Base de données

1. Le frontend envoie une requête avec le token Sanctum dans le header Authorization.
2. Laravel valide le token, applique les règles métier, interroge la base de données.
3. La réponse JSON est renvoyée au frontend qui met à jour l'interface sans rechargement.

2.1 Base de données :



Result Grid								
Filter Rows:								
Edit: Export/Import: Wrap Cell Content:								
	id	name	email	email_verified_at	password	remember_token	created_at	updated_at
▶	1	Test User	test@example.com	2026-05-07 19:10:33	\$2y\$12\$e6kiyWxGNP8xsuIc4WQZ4ueCKeffTk...	QIZKNXAwbu	2026-05-07 19:10:35	2026-05-07 19:10:35
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3. Caputres commentées

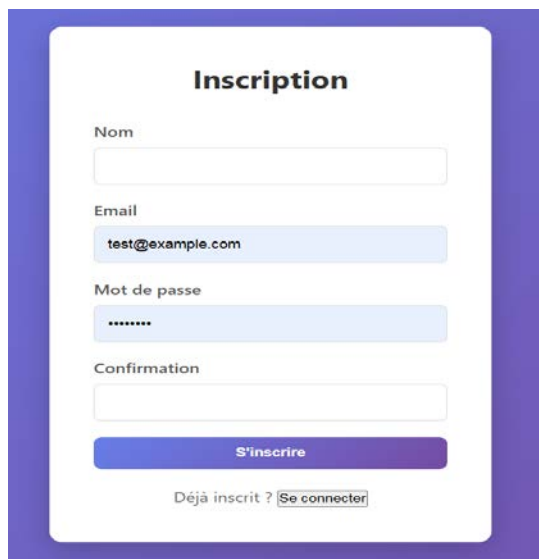
Page de connexion :

A screenshot of a login page titled 'Connexion'. The page has a white background with a purple border. It contains the following elements:

- Connexion**: The title of the page.
- Email**: A label for the email input field.
- test@example.com**: The text entered in the email input field.
- Mot de passe**: A label for the password input field.
-**: The text entered in the password input field, represented by dots.
- Se connecter**: A button to submit the login form.
- Pas encore inscrit ?**: A link to the registration page.
- S'inscrire**: A button to register.

Commentaire : Cette interface permet à un utilisateur déjà inscrit d'accéder à son espace personnel de gestion de notes. Lorsque l'utilisateur clique sur le bouton "se connecter " les informations sont envoyées à L'API Laravel via une requête AJAX avec Axios , si les données sont correctes, Laravel Sanctum génère un token d'authentification permettant l'accès sécurisé aux fonctionnalités de l'application.

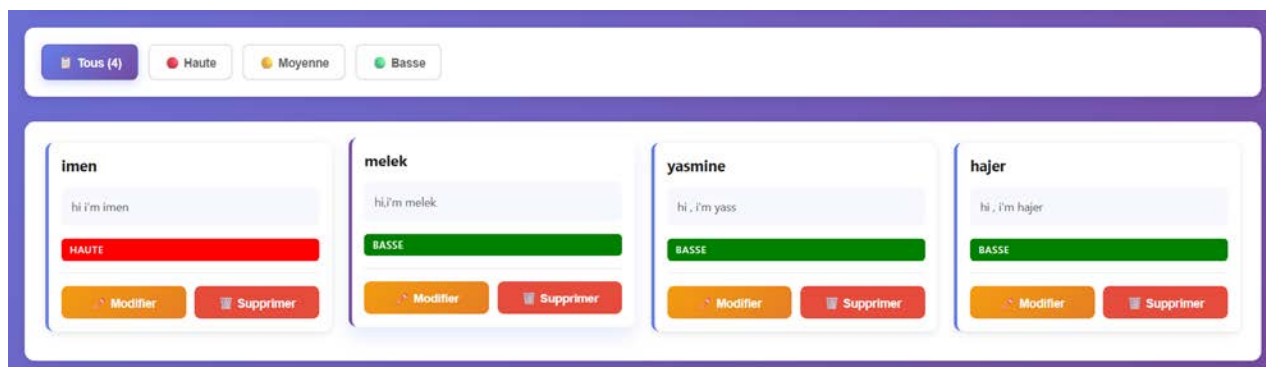
Page d'inscription :



The image shows a registration form titled "Inscription". It contains four input fields: "Nom", "Email" (with the placeholder "test@example.com"), "Mot de passe" (with masked characters "*****"), and "Confirmation". Below the fields is a blue button labeled "S'inscrire". At the bottom, there is a link "Déjà inscrit ? [Se connecter](#)".

Commentaire : Cette interface permet à un nouvel utilisateur de créer un compte afin d'accéder à l'application de gestion de notes personnelles. Après validation des informations, un nouvel utilisateur est enregistré dans la base de données.

Liste de notes :



The image shows a dashboard for managing notes. At the top, there is a filter bar with buttons: "Tous (4)", "Haute" (red), "Moyenne" (yellow), and "Basse" (green). Below the filter bar, there are four note cards, each representing a user: "imen", "melek", "yasmine", and "hajer". Each card displays a message (e.g., "hi i'm imen"), a status bar (e.g., "HAUTE" in red for imen, "BASSE" in green for others), and two buttons: "Modifier" (orange) and "Supprimer" (red).

Commentaire : Cette interface représente la page principale de l'application après l'authentification de l'utilisateur.

Elle permet d'afficher toutes les notes personnelles enregistrées dans la base de données sous forme de cartes organisées et lisibles.

Ajout d'une note :



The form is titled '+ Créer une note'. It contains three main input areas: a text field for 'Titre *' with the placeholder 'Titre...', a larger text area for 'Description' with the placeholder 'Écris ta note ici...', and a dropdown menu for 'Priorité' currently set to 'Basse' with a green circle icon. At the bottom is a blue button labeled 'Ajouter'.

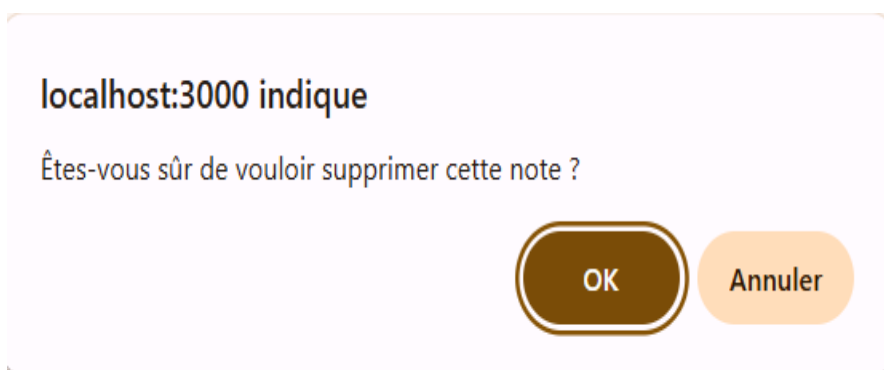
Commentaire: L'interface de création de notes permet à l'utilisateur d'ajouter facilement une nouvelle note. Elle est composée d'un formulaire simple contenant plusieurs champs essentiels : un champ pour saisir le **titre** de la note, une zone de texte pour écrire la **description**, ainsi qu'un champ permettant de définir la **priorité** (par exemple : faible, moyenne ou élevée).

Modification d'une note :



The form is titled 'Modifier la note' with a pencil icon. It contains three main input areas: a text field for 'Titre *' with the value 'melek', a larger text area for 'Description' with the value 'i'm melek', and a dropdown menu for 'Priorité' currently set to 'Haute' with a red circle icon. At the bottom is a blue button labeled 'Mettre à jour'.

Suppression d'une note :



The dialog has a light purple background and contains the text 'localhost:3000 indique' followed by the question 'Êtes-vous sûr de vouloir supprimer cette note ?'. At the bottom are two buttons: 'OK' in a dark blue rounded rectangle and 'Annuler' in a light blue rounded rectangle.

Note supprimée avec succès !

4. Extraits de Code

4.1 NoteController@store (Laravel)

```
public function store(Request $request)
{
    $request->validate([
        'title' => 'required|string|max:255',
        'content' => 'nullable|string',
        'priority' => 'nullable|in:basse,moyenne,haute',
    ]);

    return Note::create([
        'title' => $request->title,
        'content' => $request->content ?? '',
        'priority' => $request->priority ?? 'basse',
        'user_id' => $request->user()->id,
    ]);
}
```

4.2 NoteForm avec Axios (React)

```
import { useState, useEffect } from "react";
import API from "../api/axios";

const INITIAL_STATE = { title: "", content: "", priority: "basse" };
const PRIORITIES = [
    { value: "basse", label: "● Basse" },
    { value: "moyenne", label: "● Moyenne" },
    { value: "haute", label: "● Haute" },
];

function NoteForm({ fetchNotes, editingNote, onEditComplete, showNotification }) {
    const [form, setForm] = useState(INITIAL_STATE);
    const [error, setError] = useState("");
    const [loading, setLoading] = useState(false);

    useEffect(() => {
        setForm(editingNote || INITIAL_STATE);
    });
}
```

```

    setError("");
  }, [editingNote]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm((prev) => ({ ...prev, [name]: value }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!form.title.trim()) {
      setError("Le titre est obligatoire.");
      return;
    }

    setLoading(true);

    try {
      if (editingNote) {
        await API.put(`/notes/${editingNote.id}`, form);
      } else {
        await API.post("/notes", form);
      }

      fetchNotes();
      onEditComplete?.();
      setForm(INITIAL_STATE);
      showNotification?.("Succès !", "success");
    } catch (err) {
      setError("Erreur API");
      console.error(err);
    } finally {

```

```
    setLoading(false);
  }
};

return (
  <div className="note-form-container">
    <form onSubmit={handleSubmit} className="note-form">
      <input
        name="title"
        value={form.title}
        onChange={handleChange} />
      <textarea
        name="content"
        value={form.content}
        onChange={handleChange}
      />
      <select
        name="priority"
        value={form.priority}
        onChange={handleChange}
      >
        {PRIORITIES.map((p) => (
          <option key={p.value} value={p.value}>
            {p.label}
          </option>
        ))}
      </select>
      <button disabled={loading}>
        {editingNote ? "Update" : "Add"}
      </button>
    </form>
  </div>
);
```



```
</form>
```

```
</div>
```

```
);
```

4.3 Configuration CORS (config/sanctum.php)

```
<?php
```

```
use Illuminate\Cokie\Middleware\EncryptCookies;
```

```
use Illuminate\Foundation\Http\Middleware\ValidateCsrfToken;
```

```
use Laravel\Sanctum\Http\Middleware\AuthenticateSession;
```

```
use Laravel\Sanctum\Sanctum;
```

```
return [
```

```
'stateful' => explode(',', env('SANCTUM_STATEFUL_DOMAINS', sprintf(
```

```
    '%s%s',
```

```
    'localhost,localhost:3000,127.0.0.1,127.0.0.1:8000,::1',
```

```
    Sanctum::currentApplicationUrlWithPort(),
```

```
    // Sanctum::currentRequestHost(),
```

```
))),
```

```
'guard' => ['web'],
```

```
'expiration' => null,
```

```
'token_prefix' => env('SANCTUM_TOKEN_PREFIX', ''),
```

```
'middleware' => [
```

```
    'authenticate_session' => AuthenticateSession::class,
```

```
    'encrypt_cookies' => EncryptCookies::class,
```

```
    'validate_csrf_token' => ValidateCsrfToken::class,
```

```
];]
```

5. Tests Manuels

Action testée	Résultat attendu	Résultat obtenu
Inscription avec données valides	Compte créé, redirection /login	Conforme
Connexion valide	Token stocké, accès /notes	Conforme
Connexion invalide	Message d'erreur 401	Conforme

Action testée	Résultat attendu	Résultat obtenu
Ajout d'une note	Note apparaît en tête de liste	Conforme
Modification d'une note	Données mises à jour immédiatement	Conforme
Suppression avec confirmation	Note disparaît de la liste	Conforme
Filtre par priorité	Seules les notes filtrées affichées	Conforme
Accès sans token	Redirection vers /login (401)	Conforme
Titre vide à la soumission	Soumission bloquée côté client	Conforme

6. Difficultés Rencontrées & Solutions

Difficulté	Cause	Solution
Erreurs CORS	Le navigateur bloquait les requêtes cross-origin vers l'API	Configuration de config/cors.php avec les origines autorisées et supports_credentials: true
Gestion du token	Le token Sanctum devait être transmis à chaque requête protégée	Stockage dans localStorage, injection automatique via Axios interceptors ou headers manuels
Mise à jour de la liste	Après un ajout ou une suppression, la liste ne se rafraîchissait pas automatiquement	Appel systématique de fetchNotes() dans un useEffect déclenché après chaque action CRUD
Validation côté client	Le formulaire pouvait être soumis avec un titre vide	Ajout d'une validation HTML5 (required) et d'une vérification JavaScript avant l'envoi Axios

7. Bilan & Perspectives

7.1 Compétences développées

- Conception et implémentation d'une API REST complète avec Laravel (routes, contrôleurs, Eloquent, validation, Sanctum).
- Développement d'une SPA React avec gestion d'état (useState, useEffect, useContext), routing (React Router) et communication AJAX (Axios).
- Débogage de problèmes de configuration cross-domain (CORS) et sécurité des tokens.
- Production d'une interface responsive et accessible avec des retours visuels clairs (toasts, badges colorés).

7.2 Amélioration

Amélioration	Valeur ajoutée
Export PDF des notes	Possibilité de partager ou archiver ses notes hors de l'application
Mode sombre (Dark Mode)	Amélioration de l'ergonomie et accessibilité visuelle
Pagination ou défilement infini	Nécessaire lorsque le nombre de notes devient important

Ce projet a constitué une expérience formatrice complète : de la modélisation de la base de données

jusqu'au déploiement d'une interface React connectée à une API sécurisée. Il a mis en évidence l'importance de la rigueur dans la gestion des tokens, la configuration CORS, et la réactivité de l'UI.